Olist Marketplace Intelligence: Solution Approach

PROBLEM STATEMENT LINK: https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce

Business Problem and Scenario

Business scenario

- Olist is a Brazilian marketplace that connects small sellers to large retail sites. The marketplace quality team spots orders and sellers heading for trouble and applies the right remedy within policy.
- Today the work is manual: dashboard → eyeball bad reviews → guess the seller → search in a policy document.
- Each case takes about **15 minutes**, the **policy step is often skipped**, the team acts only **after** the bad review, and capacity is about **400 orders per week**.

Problem statement

- Build one system that answers an analyst's plain-language question, for example: *"Order X had a 9-day delay in furniture. What is the risk of a 1-star review, which seller is behind it, how have they performed lately, and what may we offer?"*
- The answer must be grounded in **order data**, **seller relationships** and **written policy**, and never in the model's own knowledge.

Prediction task: At order approval, predict whether the order will get a review score of 1 or 2, using only information known at that moment.

Success measures

- Cut case time from about 15 minutes to seconds.
- Flag the riskiest orders within the 400-per-week cap, with expected net benefit
- Every policy answer cites a clause and says "not found" when the policy is silent.

Roadmap

Part 1. Exploration and data quality: Profile 9 tables, join safely to one row per order, mark which columns are known at approval.
Part 2. Risk model: Build leakage-safe features, train baseline and gradient boosting models with time-based validation, and set a threshold for 400 orders per week.
Part 3. Policy retrieval (RAG): Chunk the policy by clause, return cited answers, and abstain when the policy is silent.
Part 4. Knowledge graph: Link customers, orders, sellers and products, and extract complaint types from Portuguese reviews.
Part 5. Agent: Combine the three tools in a tool-call loop that never answers policy questions from memory.
Milestones: Data ready → Model ready → Policy ready → Graph ready → Agent ready

PROBLEM STATEMENT LINK: https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce

Raw Dataset

Sources used
Order data: The Olist dataset with 9 linked tables (about 100,000 orders). The tables are listed below.
Policy: A fictional marketplace policy with 8 sections and 33 clauses (MOP-1 to MOP-8). It is the only source of rules.

The raw dataset consists of 9 tables covering orders, items, customers, products, payments, reviews, sellers, category translation and geographical information.

Table	Rows	Columns	Column Names
Orders	99,441	8	order_id, customer_id, order_status, order timestamps
Items	112,650	7	order_id, product_id, seller_id, shipping_limit_date, price, freight_value
Payments	103,886	5	payment_type, payment_installments, payment_sequential, payment_value
Reviews	99,224	7	review_score, review comments, review timestamps
Customers	99,441	5	customer_id, customer_unique_id, ZIP prefix, city, state
Sellers	3,095	4	seller_id, ZIP prefix, city, state
Products	32,951	9	category, name/description lengths, photo count, weight, dimensions
Geolocation	1,000,163	5	ZIP prefix, latitude, longitude, city, state
Category Translation	71	2	Portuguese to English category data

CODE LINK: https://github.com/sandipanpaul21/olist-marketplace-intelligence/blob/main/notebooks/EDA/eda_00_raw_data.ipynb

Why In-Depth EDA Before Model Building?

In-depth EDA helps identify data quality issues, unusual patterns, and unreliable features before model building. Few Examples from this dataset only:

1. Extreme Time Differences *(Reference: EDA Part 01 — Orders Table)*
- Approval time: median **0.34 hours**, maximum **188 days**. Delivery time: median **10.2 days**, maximum **209.6 days**.
- **Inference:** A delivery time of 209 days is unusually long and needs investigation before using this feature in modelling.

2. Suspicious Shipping Deadlines *(Reference: EDA Part 03 — Order Items Table)*
- Four item lines across three orders have shipping deadlines in 2020, although order data ends in 2018.
- **Inference:** These dates may be data-entry errors and require verification.

3. Extreme Review Response Times *(Reference: EDA Part 05 — Reviews Table)*
- Median response time: 1.67 days; maximum: **518.7 days**. A total of 172 responses took over 100 days.
- **Inference:** Extremely late responses need investigation, even though their timestamps follow the expected sequence

4. Product Data Quality *(Reference: EDA Part 06 — Products Table)*
- Four products have zero weight, which is physically implausible. One product weighs 40,425 g but measures only 13 × 65 × 28 cm.
- **Inference:** Zero weights should be treated as unknown. Extreme values should be investigated before deciding whether robust handling is needed.

5. Seller ZIP Prefix and State Inconsistencies *(Reference: EDA Part 07 — Seller Data)*
- 17 ZIP prefixes covering 45 sellers are associated with multiple states. Example: ZIP prefix 31160 maps Belo Horizonte to both MG and SP.
- 25 city values contain symbols or extra location text, such as sao paulo / sao paulo.
- **Inference:** Seller location fields may be inconsistent and require cleaning before geographical analysis.

6. Geolocation Data Quality *(Reference: EDA Part 08 — Geolocation Data)*
- 261,831 rows (26.2%) are fully duplicated.
- Coordinates extend beyond Brazil's expected geographic boundaries, but 33 rows (0.004%), across 21 ZIP prefixes, fall outside the expected bounding box.
- **Inference:** Deduplication can reduce data volume, while the small number of out-of-bound coordinates should be checked before geographical analysis.

Therefore, we conducted in-depth, table-level EDA on each dataset to understand its structure, data quality, distributions, patterns, and inconsistencies before proceeding with model building.

Exploratory Data Analysis (Part 1)

EDA Part 01: Orders Table — EDA and Data Quality

1. Dataset Overview

- 99,441 rows × 8 columns: order_id, customer_id, order_status, order_purchase_timestamp, order_approved_at, order_delivered_carrier_date, order_delivered_customer_date, order_estimated_delivery_date.
- Purchase dates range from 2016-09-04 to 2018-10-17.
- order_id is unique, with no duplicate rows. Each row represents one order. customer_id is also unique because a new ID is created for every order.
- **Normal journey of an order**: purchase, approval, handoff to the carrier, delivery, and the estimated delivery date promised to the customer.

2. Order Status

- 96,478 orders (97.0%) are delivered.
- The remaining 2,963 orders include shipped (1,107), canceled (625), unavailable (609), invoiced (314), processing (301), created (5), and approved (2).

3. Missing Values

- order_approved_at: 160 missing (0.16%). order_delivered_carrier_date: 1,783 missing (1.79%). order_delivered_customer_date: 2,965 missing (2.98%).
- 146 orders have all three timestamps missing. Some orders marked delivered also have missing timestamps.

4. Order Volume and Timing

- Monthly orders peaked at 7,544 in November 2017. The beginning and end of the dataset have fewer orders.
- Monday is the busiest day (16,196 orders), while Saturday is the quietest (10,887).
- The busiest time block is 12:00–15:59 (25,536 orders); the quietest is 04:00–07:59 (2,127 orders). The dataset does not specify the time zone.

5. Timestamp Consistency

- 1,359 orders have order_delivered_carrier_date before order_approved_at. Of these, 166 also have carrier handoff before purchase.
- Another 23 orders have order_delivered_customer_date before order_delivered_carrier_date.
- Inference: These timestamp sequences are inconsistent and may affect delivery-time calculations.

6. Approval and Delivery Time

- Time from order_purchase_timestamp to order_approved_at has a median of 0.34 hours, a 99th percentile of 90 hours, and a maximum of 4,509 hours (188 days).
- Time from order_purchase_timestamp to order_delivered_customer_date has a median of 10.2 days, a 95th percentile of 29.3 days, and a maximum of 209.6 days.
- Inference: Most orders are approved quickly, but some have extreme delays. These records should be checked before using the time differences in analysis or modelling.

CODE LINK: https://github.com/sandipanpaul21/olist-marketplace-intelligence/blob/main/notebooks/EDA/eda_01_orders.ipynb

EDA Part 02: Customers Table — EDA and Data Quality

1. Dataset Overview

- 99,441 rows × 5 columns: customer IDs, ZIP prefix, city, and state. No missing values or duplicate rows. customer_id is unique.
- There are 96,096 distinct customer_unique_id values, meaning some people have multiple customer IDs.

2. Customer Repeat Behaviour

- 93,099 people (96.9%) placed one order.
- 2,997 people (3.1%) are repeat customers, accounting for 6,342 orders (6.4% of all orders). One person placed a maximum of 17 orders.
- Inference: Use customer_unique_id to identify repeat customers. customer_id links an order to its customer record, not necessarily to a unique person.

3. Geographic Distribution

- Customers span 27 states. São Paulo (SP) accounts for 42.0% of records; SP, RJ, and MG together account for 66.6%.
- There are 4,119 distinct cities. São Paulo city has 15,540 customers, while 1,144 cities (27.8%) have only one customer.

4. Address Consistency

- Among repeat customers, 250 used multiple ZIP prefixes and 39 used multiple states.
- Inference: Addresses can vary across orders because customers may move or use different delivery addresses. Avoid assuming each person's address is fixed.

CODE LINK: https://github.com/sandipanpaul21/olist-marketplace-intelligence/blob/main/notebooks/EDA/eda_02_customers.ipynb

EDA Part 09: Category Translation Table— EDA and Data Quality

1. Dataset Overview

- 71 rows × 2 columns mapping Portuguese category names to English.
- No missing values or duplicate rows. Both columns contain unique values, forming a one-to-one mapping.

2. Translation Quality

- Seven category names are identical in both languages, such as audio, pet_shop, and consoles_games. These are not translation gaps.
- Three spelling errors affect four English category names: fashio, costruction (two names), and confort.
- Differences such as sports/sport, drink/drinks, and musical/musicals are valid word-form variations.
- seguros_e_servicos maps to security_and_services, although seguros usually means insurance.
- Inference: The table is largely clean, but some translations contain spelling errors or possible meaning mismatches. Flag these for review rather than changing them without confirmation.

CODE LINK: https://github.com/sandipanpaul21/olist-marketplace-intelligence/blob/main/notebooks/EDA/eda_09_category_translation.ipynb

EDA Part 03: Order Items Table — EDA and Data Quality

1. Dataset Overview

- 112,650 rows × 7 columns: order ID, item number, product, seller, shipping deadline, price, and freight.
- Each row represents an item within an order, not a complete order.
- No missing values or duplicate rows. (order_id, order_item_id) uniquely identifies each item line.
- The table contains 98,666 distinct orders, 32,951 products, and 3,095 sellers.

2. Order and Seller Distribution

- 88,863 orders (90.1%) have one item line; 9,803 (9.9%) have multiple lines.
- The largest order has 21 item lines.
- 97,388 orders (98.7%) have a single seller; 1,278 (1.3%) involve multiple sellers.

3. Price and Freight

- Median price: 74.99; mean: 120.65; 99th percentile: 890; maximum: 6,735.
- Median freight: 16.26; mean: 19.99; maximum: 409.68.
- No prices are zero or negative. 383 lines (0.34%) have free freight, while 4,124 (3.7%) have freight higher than the item price.

4. Shipping Deadline

- Most deadlines fall in 2017 (49,765 lines) and 2018 (62,511 lines).
- Four lines across three orders have deadlines in 2020, even though the orders data ends in 2018.
- Inference: The 2020 deadlines are suspicious and may be data-entry errors. They should be checked rather than automatically corrected.

5. Data Consistency

- The orders table contains 99,441 rows, while this table covers 98,666 distinct orders—a difference of 775 orders.
- Inference: Some orders may have no item lines. Check these orders during the join to understand why they are missing.

CODE LINK: https://github.com/sandipanpaul21/olist-marketplace-intelligence/blob/main/notebooks/EDA/eda_03_items.ipynb

EDA Part 04: Payments Table — EDA and Data Quality

1. Dataset Overview
- 103,886 rows × 5 columns: order ID, payment sequence, payment type, installments, and payment value.
- No missing values or duplicate rows. (order_id, payment_sequential) uniquely identifies each payment line.

2. Multiple Payments
- 96,479 orders (97%) have one payment line, while 2,961 orders (3%) have multiple lines.
- The largest order has 29 payment lines, including several small voucher payments.
- Inference: One order can have multiple payments, so payment values must be added together to calculate the total order value.

3. Payment Methods
- Credit card: 73.9%; boleto: 19.0%; voucher: 5.6%; debit card: 1.5% of payment lines.
- Of 2,246 orders using multiple payment types, 2,245 combine credit cards and vouchers.
- 3,866 orders use vouchers; 1,621 are paid entirely by voucher.
- Inference: Vouchers are often used to pay part of an order alongside a credit card.

4. Installments
- 50.6% of payment lines have one installment; the maximum is 24.
- 66.9% of credit card payments use multiple installments, with a median of 3.
- Two credit card payments have zero installments. Since a card payment must have at least one installment, these values are invalid.
- Inference: Installment count can help describe customer payment behavior, but invalid values should be checked before modelling.

5. Payment Values and Unusual Records
- 1,150 payment lines exceed 1,000, and six exceed 5,000.
- The highest payment is 13,664.08 by credit card in one installment. These high values may reflect expensive purchases and should not automatically be removed.
- Three not defined payment lines have zero value, indicating an unknown payment type rather than a valid payment method.
- Inference: Use total payment value and payment method as potential model features, while checking unusual records for data quality issues.

CODE LINK: https://github.com/sandipanpaul21/olist-marketplace-intelligence/blob/main/notebooks/EDA/eda_04_payments.ipynb

EDA Part 05: Reviews Table — EDA and Data Quality

1. Dataset Overview
- 99,224 rows × 7 columns: Review ID, order ID, review score, comment title, comment message, creation date, and answer timestamp.
- Review scores range from 1 to 5. Comment fields contain free text in Brazilian Portuguese.
- Review dates range from 2016-10-02 to 2018-08-31, while answers extend to 2018-10-29. Orders span 2016-09-04 to 2018-10-17.

2. Duplicates and Multiple Reviews
- No, fully duplicated rows; (review_id, order_id) is unique.

3. Missing Values
- Review title missing: 87,656 (88.34%); review message missing: 58,247 (58.70%).
- 27 messages contain only spaces.
- 56,536 reviews have neither title nor message.
- Decision: Treat blank comments as missing and do not impute review text.

4. Review Score Distribution
- 5 stars: 57.8%, 4 stars: 19.3%, 3 stars: 8.2%, 2 stars: 3.2%, 1 star: 11.5%.
- The distribution is U-shaped, with most reviews receiving 5 or 1 star. No invalid scores were found.

5. Missing Comments and Sentiment
- Messages are present in 76.5% of 1-star reviews versus 35.8% of 5-star reviews. Median message lengths are 89 and 42 characters, respectively.
- Common positive phrases include muito bom, bom, and recomendo.
- nao recebi (not received) appears in 16.7% of bad-review comments versus 0.3% of good-review comments. Other complaints include delays, defects, wrong items.

6. Review Response Time
- Median response time: 1.67 days; 95th percentile: 6.97 days; 99th percentile: 22.07 days.
- 682 responses took over 30 days, including 172 over 100 days. Maximum: 518.7 days.
- No responses occurred before review creation. Median response times are similar across scores (1.55–1.79 days).
- The only extreme values are late answers: 172 after more than 100 days, up to 518.7 days. The answer date looks valid, so the customer simply answered very late.

CODE LINK: https://github.com/sandipanpaul21/olist-marketplace-intelligence/blob/main/notebooks/EDA/eda_05_reviews.ipynb

EDA Part 06: Products Table — EDA and Data Quality

1. Dataset Overview
- 32,951 rows × 9 columns: Product ID, category, listing attributes (name length, description length, photos), weight, and three dimensions.
- Two source column names are misspelled: product_name_lenght and product_description_lenght.
- Decision: Rename them to product_name_length and product_description_length in the working dataset.

2. Missing Values
- 610 products (1.85%) have missing category, name length, description length, and photo count, indicating incomplete product listings.
- Decision: Retain these products and handle missing values appropriately during preprocessing rather than dropping them.

3. Invalid and Extreme Values
- Four products have 0 g weight, which is physically implausible. All belong to cama_mesa_banho and share dimensions of 30 × 25 × 30 cm.
- The heaviest product weighs 40,425 g but has dimensions of only 13 × 65 × 28 cm, suggesting a potential data inconsistency (Explained in the last section).
- Decision: Treat zero weight as unknown. Retain extreme values initially and apply capping or other robust handling within the model pipeline, if justified by the feature distributions.

4. Feature Relationships
- Product weight and box volume have a correlation of 0.80, indicating a strong positive relationship, which is reasonable for physical products.
- Decision: Retain both features, while checking for outliers and potential redundancy during feature engineering.

5. Category Translation
- Seven category names are identical in Portuguese and English, including audio, cool_stuff, pet_shop, and market_place.
- Decision: Keep these mappings unchanged because the terms are already used in English; identical translations do not necessarily indicate errors.

Why this is suspicious:
- Weight: 40,425 g (about 40 kg).
- Box size: 13 × 65 × 28 cm, with a volume of about 24 litres.
- A 40 kg product in such a small box is unusual for common marketplace products like clothes, toys, or household items.
- The weight may have been entered incorrectly, possibly due to an extra digit. For example, 4,042 g (about 4 kg) would be more reasonable for this box size.
- However, the product could be a heavy item, such as metal parts or tools, or the dimensions could be incorrect.

CODE LINK: https://github.com/sandipanpaul21/olist-marketplace-intelligence/blob/main/notebooks/EDA/eda_06_products.ipynb

EDA Part 07: Seller Data — EDA and Data Quality

1. Dataset Overview

- 3,095 rows × 4 columns: Seller ID, ZIP code prefix, city, and state.
- Each row represents one seller. Product details and seller performance must be derived by joining other tables.
- seller_id is unique, with no duplicate rows or missing values.

2. Geographic Distribution

- Sellers operate across 23 states.
- São Paulo (SP) accounts for 1,849 sellers (59.7%); the top five states (SP, PR, MG, SC, RJ) account for 90.6%.
- Five states (AC, PI, MA, AM, PA) have only one seller each.
- There are 611 distinct city names. Top cities include São Paulo (694 sellers), Curitiba (127), Rio de Janeiro (96), Belo Horizonte (68), and Ribeirão Preto (52).
- 342 cities (56.0%) have only one seller, indicating a long-tailed distribution.

3. City Name Quality

- City names are mostly lowercase with no extra spaces. Three values contain accents; removing accents reduces distinct names from 611 to 609.
- One city value (04482255) appears to be a ZIP code mistakenly entered as a city.
- Twenty-five city values contain symbols or additional location text, e.g., sao paulo / sao paulo and novo hamburgo, rio grande do sul, brasil.
- Thirty-four ZIP prefixes have multiple city-name variants, including spelling errors such as garulhos / guarulhos and sando andre / santo andre.

Decision: Do not use city names as a reliable join key. Avoid manual corrections based on assumptions; use ZIP prefixes for location mapping.

4. ZIP Prefix and State Inconsistencies

- 17 ZIP prefixes (45 sellers) are associated with multiple states, suggesting incorrect state assignments.
- Examples:
  - ZIP prefix 21210: Rio de Janeiro city mapped to both RJ and RN.
  - ZIP prefix 22783: Rio de Janeiro city mapped to both RJ and SP.
  - ZIP prefix 31160: Belo Horizonte city mapped to both MG and SP.
- Twenty-three city names appear across multiple states, partly reflecting these inconsistencies.
- Customers span 27 states, with approximately 42% in SP, indicating potential cross-state orders.

Decision: Retain all seller records because there are no missing or duplicate rows. Treat seller_state as potentially unreliable and cross-check it against the cleaned geolocation lookup after joining on ZIP prefix.

CODE LINK: https://github.com/sandipanpaul21/olist-marketplace-intelligence/blob/main/notebooks/EDA/eda_07_sellers.ipynb

EDA Part 08: Geolocation Data — EDA and Data Quality

1. Dataset Overview
- 1,000,163 rows × 5 columns: ZIP code prefix, latitude, longitude, city, and state.
- Largest table, containing approximately 10× more rows than the orders table.
- Each row represents a coordinate point, not a unique ZIP code prefix. Multiple points can exist for the same prefix.

2. Duplicate Analysis
- Fully duplicated rows: 261,831 (26.2%).
- Rows remaining after exact deduplication: 738,332.
- Repeated (prefix, latitude, longitude) combinations: 280,009.
- Distinct (prefix, latitude, longitude) combinations: 720,154.

3. Missing Values and Geographic Validity
- No missing values were found in any column.
- Latitude ranges from −36.61 to 45.07 and longitude from −101.47 to 121.11, exceeding Brazil's geographic boundaries.
- Only 33 rows (0.004%), across 21 prefixes, fall outside the expected Brazil bounding box.
- Decision: Exclude points outside Brazil from the geolocation lookup because they are geographically invalid and account for a negligible share of the data.

4. City Name Quality
- Non-ASCII city text: 63,436 rows (8.6% after exact deduplication).
- Distinct city names: 8,011 raw, reduced to 5,967 after accent removal.
- São Paulo appears under two spellings: sao paulo (79,927 rows) and são paulo (19,718 rows).
- Nineteen city names contain symbols, digits, or brackets; two contain web-encoded characters.
- 8,556 prefixes (45.0%) have more than one city name, indicating inconsistent naming.
- Decision: Ignore city names as a reliable join key because of spelling, accent, and encoding inconsistencies.

5. State Consistency
- Eight ZIP code prefixes appear under multiple states, indicating incorrect state assignments.
- No invalid state codes were identified.
- Decision: Assign each prefix its most frequently observed state and use this as the state lookup.

CODE LINK: https://github.com/sandipanpaul21/olist-marketplace-intelligence/blob/main/notebooks/EDA/eda_08_geolocation.ipynb

Retrieval Augmented Generation (Part 3)

Marketplace Policy Assistant (RAG): Answers You Can Trace to a Clause

Policy RAG: answers that are cited and honest
The problem: Staff need policy answers in seconds, not 15 minutes of searching. A wrong answer is worse than no answer. Every answer must cite a clause, and the system must say "not found" when the policy is silent.

The data: A fictional marketplace policy: 8 sections, 33 clauses (MOP-1 to MOP-8). 20 test questions: 16 have an answer in the policy, 4 do not.

NOTE: **MOP** = Marketplace Operations Policy. It comes from the title of Appendix A in our data brief: "Marketplace Operations Policy". MOP-1 to MOP-8 are its 8 sections, so the letters are the document's short name and the number is the section.

The 5 steps and the question each one answers

1. **Chunking:** how do we cut the policy so every piece can be cited?
    - One chunk per clause, with an ID such as `MOP-6.4`
    - Why not one whole chunk? (Hint: logical chunking)

2. **Embedding and retrieval:** can we find the right clause for a question?
    - Local embeddings (meaning) and BM25 (words), combined
    - Why not one search only? (Hint: it misses exact terms or different wording)

3. **Retrieval evaluation:** how often is the right clause found, before any LLM?
    - 20 questions, measured with recall@1, @3 and @5
    - Why not judge final answers only? (Hint: a retrieval error and an LLM error look the same)

4. **Generation:** can the LLM answer only from those clauses, with citations?
    - Top 3 clauses go in, a JSON answer comes out
    - Why not a free answer? (Hint: the LLM fills gaps with rules it invents)

5. **Faithfulness and abstention:** is the answer supported, and does it say "not found" when it should?
    - Number and judge checks
    - Why not trust the LLM alone? (Hint: it can still answer a question the policy does not cover)

DATA LINK: https://github.com/sandipanpaul21/olist-marketplace-intelligence/blob/main/notebooks/RAG/policy/marketplace_policy.txt

Policy RAG — Step 1: Policy Chunking

1. Objective

- Split the marketplace policy into meaningful chunks, each with a unique ID for citation. Reusable implementation: `src/policy_rag.py`.

2. Chunking Strategy

- Create one chunk per policy clause, such as `MOP-4.1`. A new chunk starts when a line begins with a clause number like `4.1` or `6.4`.
- Bullets, tables, numbered lists, and continuation lines remain within the current clause.
- Example: Clause `MOP-4.1` contains the refund rule and all four eligibility conditions in one chunk. This prevents retrieving the rule without its conditions.

3. Chunking Results

- 33 chunks across 8 sections, with 3–5 clauses per section. Five clauses contain tables or lists and remain intact.
- No further splitting is needed because each clause is already a meaningful unit.

4. Cross-References

- 10 of 33 clauses refer to other clauses or sections. Example: `MOP-8.4` references `MOP-2.2` and `MOP-2.4`.
- Automatic retrieval of referenced clauses can be considered if evaluation shows incomplete answers.
- Inference: Clause-based chunking preserves rules and their supporting conditions, enabling more complete retrieval and citation-grounded answers.

Background: Chunking in RAG

1. **Why Chunking Is Needed: Context window limit:** Large documents may not fit into the LLM prompt**. Better retrieval:** Smaller, focused chunks improve the relevance of retrieved information. **Flow**: Document → Chunking → Chunks → Embeddings → Vector DB → Retrieval → LLM

2. Chunk Size and Overlap: Chunk size: Maximum text length, measured in characters or tokens. **Overlap**: Repeats some text between adjacent chunks to preserve context. **Trade-off:** More overlap preserves context but increases storage and embedding costs.

3. Chunking Strategy by Content: FAQ: One question and answer per chunk; no overlap. **SOP**: Split by steps or sections; overlap steps if needed. **Policy**: Preserve complete clauses, conditions, and related tables. **Knowledge articles**: Use sentence-aware or semantic chunking with suitable overlap.

4. Common Text Splitters: CharacterTextSplitter: Splits using a specified separator. **RecursiveCharacterTextSplitter:** Tries multiple separators, from paragraphs to smaller units, before splitting further. **TokenTextSplitter:** Splits based on token count. **MarkdownHeaderTextSplitter:** Splits using Markdown headings.

Key takeaway: Choose chunking based on document structure. For policy RAG, preserving complete clauses and their conditions is more important than applying a fixed chunk size or overlap.

CODE LINK: https://github.com/sandipanpaul21/olist-marketplace-intelligence/blob/main/notebooks/RAG/rag_01_policy_chunking.ipynb

Policy RAG — Step 2: Embedding and Retrieval (Part 1)

1. Objective: Convert 33 policy clauses into vectors and test the `search` function using `dense`, `bm25`, and `hybrid` retrieval.

2. Embedding Model: `bge-small-en-v1.5`

- The code uses `sentence-transformers` to load the embedding model and generate vectors.
- BGE = BAAI General Embedding; small = lightweight model (~33M parameters); en = English; v1.5 = model version.
- The model generates 384 numbers per text and is downloaded from Hugging Face on first use. The library and model are different: `sentence-transformers` is the library, while BGE is the model.
- The model runs locally after download, avoiding the need to send policy text to an external embedding API.

3. Embedding Matrix: `(33, 384)`

- The code generates embeddings for all 33 clauses, producing a matrix with 33 rows and 384 columns.
- 33 rows: one vector per clause. 384 columns: the numerical representation of each clause. • Total: `33 × 384 = 12,672` numbers.
- The vectors are stored in `index.emb`, and each row maps by position to `chunks[i]`, allowing the code to recover the corresponding clause ID.
- Example: If a simplified vector for `MOP-4.2` were `[0.6, 0.8, 0.0]`, those three numbers together would represent its meaning. The actual model produces 384 numbers, not three.

4. How the Search Code Works

- The code embeds the user's question into a 384-dimensional vector.
- It compares the query vector with the 33 clause vectors. It sorts the scores and returns the top-k clause IDs.
- Example from the notebook: For “How long does a Pix refund take?”, `MOP-4.2` ranks first in dense, BM25, and hybrid search. Its reported cosine similarity is `0.825`.

5. Cosine Similarity

- Cosine similarity measures how closely two vectors point in the same direction: `1` = same direction, `0` = perpendicular, `−1` = opposite.

6. Search Modes and Results

- **Dense**: Retrieves clauses based on semantic similarity.
- **BM25**: Retrieves clauses based on keyword relevance.
- **Hybrid**: Combines dense and BM25 rankings.
- Observed results: Seller tier question: dense and hybrid rank `MOP-1.2` first; BM25 ranks `MOP-1.3` first. Pix refund timing: all three rank `MOP-4.2` first. Freight responsibility: all three rank `MOP-8.3` first.

CODE LINK: https://github.com/sandipanpaul21/olist-marketplace-intelligence/blob/main/notebooks/RAG/rag_02_embedding_retrieval.ipynb

Policy RAG — Step 2: Embedding and Retrieval (Part 2)

7. Hybrid Search: Reciprocal Rank Fusion (RRF)
- Dense and BM25 scores use different scales, so the code combines their rankings rather than directly adding raw scores.
- RRF formula: `RRF score = Σ 1/(60 + rank)`. A clause receives a contribution from each ranked list. A better rank produces a larger contribution.
- Example: A clause ranked first by both methods receives `1/61 + 1/61 = 0.0328`. RRF rewards clauses that rank well across both methods.

8. Embedding Storage
- The embeddings are stored in memory in `index.emb`; the described implementation does not save them to disk.
- With only 33 clauses, NumPy is sufficient. A vector database is not necessary for this small dataset.

Background Concepts

1. Text Representation and Tokenization
- **Text representation:** Converts text into numbers that NLP models can process.
- **Tokenization:** Splits text into words, subwords, or characters. BERT commonly uses WordPiece; GPT models commonly use Byte Pair Encoding (BPE).

2. Sparse Retrieval: TF-IDF and BM25
- **TF-IDF**: `TF × IDF`. TF measures word frequency within a document; IDF gives more weight to words that are rare across documents.
- Limitation: Repeated words can increase TF-IDF scores linearly, and long documents may receive an advantage.
- **BM25**: Improves ranking with diminishing returns for repeated terms (k_1) and document-length normalization (b).
- Limitation: BM25 relies on keyword overlap and may miss matching meaning when wording differs.

3. Dense Retrieval: Embeds queries and documents and retrieves them by semantic similarity. Limitation: A single vector compresses the text into one representation and may lose token-level detail.

4. Hybrid Search: Combines BM25 (keywords) and dense retrieval (meaning). RRF combines ranks rather than raw scores, avoiding problems caused by different score scales. Takeaway: BM25 finds matching words; dense retrieval finds related meaning; hybrid search uses both.

5. Vector Databases and Storage: A vector database stores embeddings with text and metadata for similarity search and filtering.
- **FAISS**: Vector search library; the application manages text, metadata, and persistence.
- **Chroma**: Simple embedded vector database, useful for prototypes and smaller applications.
- **Qdrant**: Supports persistence, metadata filtering, APIs, and production deployment.
- **Pinecone**: Managed cloud vector database with usage-based pricing.

CODE LINK: https://github.com/sandipanpaul21/olist-marketplace-intelligence/blob/main/notebooks/RAG/rag_02_embedding_retrieval.ipynb

Policy RAG — Step 3: Retrieval Evaluation (Part 1)

1. Objective

Measure how often the search finds the correct clause, before any LLM is used. If the right clause is not retrieved, the LLM cannot fix it.

2. Evaluation Set

- 16 answerable questions, each with its correct clause ID(s) (gold_ids). 9 need one clause and 7 need two.
- 4 unanswerable questions are not used here.
- The questions and gold clauses were written from the policy with AI help, not taken from an official set.

3. Settings Compared

- 4 methods: TF-IDF, BM25, dense (cosine similarity of embeddings), and hybrid (BM25 + dense with RRF).
- 2 text versions for each method: plain (clause only) and plain+header (section title + clause).

4. Metrics

- **any@k:** at least one correct clause is in the top k.
- **all@k:** every correct clause is in the top k. This is the main metric, because the LLM needs every clause the answer depends on.
- **MRR:** 1 divided by the rank of the first correct clause, averaged over questions. It uses the full ranking here.
- **first_rank:** the rank of the first correct clause.
- **k_needed:** the rank of the last correct clause, which is the smallest k that finds all of them.

5. Why all@1 Cannot Exceed 0.56

With k=1 only one clause is returned, so the 7 two-clause questions can never pass. The best possible all@1 is 9/16 = 0.56. Dense + header reaches it.

6. Results (leaderboard order)

- Dense + header: any@1 1.00, all@3 0.94, all@5 1.00, MRR 1.00, k_needed_max 4
- Dense plain: 0.94, 0.94, 1.00, 0.97, k_needed_max 4
- Hybrid + header: 0.81, 0.94, 1.00, 0.91, k_needed_max 5
- TF-IDF + header: 0.69, 0.81, 1.00, 0.81, k_needed_max 5
- Hybrid plain: 0.75, 0.94, 0.94, 0.86, k_needed_max 7
- BM25 + header: 0.75, 0.88, 0.94, 0.86, k_needed_max 11
- TF-IDF plain: 0.62, 0.75, 0.94, 0.76, k_needed_max 8
- BM25 plain: 0.81, 0.75, 0.88, 0.87, k_needed_max 11

CODE LINK: https://github.com/sandipanpaul21/olist-marketplace-intelligence/blob/main/notebooks/RAG/rag_03_retrieval_eval.ipynb

Policy RAG — Step 3: Retrieval Evaluation (Part 2)

7. What the Results Show

- **Dense wins.** Dense + header ranks a correct clause first for 16 of 16 questions, and finds every correct clause within the top 4.
- **The section title helps.** For dense, hybrid and TF-IDF it helped or tied on every score. For BM25 it helped at k=3 and k=5, but any@1 dropped from 0.81 to 0.75.
- **Word-based methods are weaker.** TF-IDF and BM25 rank a correct clause first for only 10 to 13 of 16 questions. They struggle when the question and the clause use different words or numbers. Q04 says "7 days late", while the clause says "4–10 days". Dense ranks that clause 1st, and the word-based methods rank it 3rd to 6th.
- **Hybrid did not beat dense.** Hybrid + header missed rank 1 on Q04, Q05 and Q09. In all three, BM25 had the same wrong top clause, so the weaker list pulled the result down.

8. Failure Analysis: Q07

- Question: R$ 500 order, 12 days late, how much credit? It needs `MOP-2.2` (12 days means Severe) and `MOP-3.2` (the credit for Severe).
- Search finds `MOP-3.2` first but ranks `MOP-2.2` lower, because the question never says "Severe". No setting finds both in the top 3. The best settings need the top 4.

9. Decision: Use dense embeddings with section title. Send the top 5 clauses to the LLM. k=4 is the minimum, and k=5 adds a safety margin.

10. Limitations and Next Steps

- Only 16 questions, written with AI help and close to the policy wording. Scores may be optimistic.
- Next: add harder questions in plain words, then test generation with 5 clauses.

Background Concepts

1. Why Evaluate Retrieval Separately: A wrong RAG answer can come from retrieval or from generation. Testing retrieval first shows which one is at fault, and it is cheaper because no LLM calls are needed.

2. Ground Truth (Gold Labels): For each question, a person marks which clauses answer it. Metrics only measure agreement with these labels, so wrong labels give wrong.

3. Hit Rate and MRR

- Hit rate (our any@k): did at least one correct result appear in the top k?
- MRR (Mean Reciprocal Rank): rewards a correct result near the top. Rank 1 gives 1.0, rank 2 gives 0.5, rank 3 gives 0.33.

4. Choosing k: A larger k raises recall but adds noise and cost. Extra clauses can distract the LLM or tempt it to cite the wrong clause. The rule used here: take the smallest k where all@k reaches 1.00 (`k_needed_max`), then add a small margin.

CODE LINK: https://github.com/sandipanpaul21/olist-marketplace-intelligence/blob/main/notebooks/RAG/rag_03_retrieval_eval.ipynb

Policy RAG — Step 4: Generation (Part 1)

1. Objective: Answer each question only from the retrieved clauses, with citations, or say "not found" when the policy is silent. The answer is only as good as the clauses the search hands over.

2. How an Answer Is Produced

- **Flow**: question, then search the top 3 clauses (hybrid, first run), then build the prompt, then the LLM answers, then the code checks the citations.
- **Model**: Claude Sonnet 5.5 (`claude-sonnet-5-5`), one call per question, 20 questions.
- **Output** is JSON with three fields: `answerable`, `answer`, `citations`.
- **Code checks**: cited IDs must be among the retrieved clauses. An answer with no valid citation, or with invalid JSON, is rejected.
- **Answers** are saved to `answers.json`, so results can be reproduced.

3. The Prompt We Gave the Model: System prompt, word for word:

You answer questions about a marketplace operations policy.
Rules:
1. Use ONLY the policy clauses given in the user message. Never use outside knowledge.
2. If the clauses do not contain the answer, set "answerable" to false. Do not guess.
If a clause says the topic is handled by another document, say that.
3. Cite every clause you used by its ID, for example MOP-2.2. Cite only IDs that were provided.
4. If the question needs a calculation, show the short working using numbers from the clauses.
Reply with JSON only, no other text, in exactly this form:
{"answerable": true or false, "answer": "...", "citations": ["MOP-1.2"]}
If answerable is false, "answer" must say briefly what the policy does not cover.

- Rule 1 stops outside knowledge. Rule 2 allows "not found". Rule 3 makes every answer traceable. The JSON format lets the code verify the answer.

4. What the Model Receives: For each question, the user message lists the retrieved clauses, each under its ID in square brackets (`[MOP-3.2]`, then the clause text), followed by `Question: <the question>`.

5. Results

- 15 of 16 answerable questions were answered, and the answers match the expected answers (checked by reading).
- 4 of 4 unanswerable questions were refused. Q18 (defective product) was refused with a pointer to the Returns Policy in `MOP-3.5`.
- Good working: Only one miss: Q07, a wrong refusal.

CODE LINK: https://github.com/sandipanpaul21/olist-marketplace-intelligence/blob/main/notebooks/RAG/rag_04_generation.ipynb

Policy RAG — Step 4: Generation (Part 2)

6. Example: Q07, Why the Model Could Not Answer

- Question: "An order worth R$ 500 is delivered 12 days late. How much platform credit does the customer get automatically?"
- Clauses received: `MOP-3.2` (credit for each delay class), `MOP-3.1` (credit is the default), `MOP-3.4` (limit on discretionary compensation).
- Clause not received: `MOP-2.2`, which says Severe means 11 to 20 days.
- Model answer (short): the clauses do not define the delay classes, so a 12-day delay cannot be placed in a class, and the credit cannot be determined.
- Correct answer: Severe, so the greater of R$ 50 or 20% of R$ 500, which is **R$ 100**.
- Verdict: the model behaved correctly and refused instead of guessing. The cause is retrieval, not the model.

7. What the Results Show

- **Retrieval limits generation.** A missing clause turns a correct answer into a refusal.
- **Refusal came from the LLM.** All 5 refusals were made by the LLM. The score threshold never fired.

8. Limitations and Next Steps

- This run used hybrid search with the top 3 clauses. The retrieval step recommended dense + header with the top 5.
- Next: re-run with dense + header and top 5, and check if Q07 is fixed and the 4 unanswerable questions are still refused.
- Then: faithfulness checks and the final abstention numbers.

Background Concepts

- **Hallucination and Abstention:** When context is missing, a model may fill the gap with a plausible but invented answer. Saying "not found" is the correct behavior, and Q07 shows it: the model refused instead of guessing the delay class.
- **Structured Output (JSON):** Asking for a fixed JSON format lets the code read the fields, check the citations and reject bad answers automatically.
- **Citation Check and Faithfulness:** A valid citation only proves the clause ID was real. It does not prove the clause supports the claim. The next step checks that with a judge, and compares the numbers in the answer with the clauses.
- **Choosing How Many Clauses to Send (k):** More clauses mean more chance of including every needed clause, but also more noise and cost. Q07 shows the cost of too few. The retrieval step chose k=5, with k=4 as the minimum.

Notes: Section 2 names Sonnet 5.5 because that is the default in the code. If you set a different `RAG_LLM_MODEL` in `.env`, change the name.

CODE LINK: https://github.com/sandipanpaul21/olist-marketplace-intelligence/blob/main/notebooks/RAG/rag_04_generation.ipynb

Policy RAG — Step 5: Faithfulness and Abstention (Part 1)

1. Objective: Check that every answer is supported by the clauses it cites, and that the system says "not found" when the policy is silent.

2. Faithfulness Checks (on each answered question)

- **Citation check:** every cited ID is a clause that was retrieved.
- **Number check:** numbers in the answer come from the cited clauses or from the question. A computed number is flagged for reading, not failed.
- **LLM judge:** compares the answer with the cited clauses and returns SUPPORTED, PARTIAL or UNSUPPORTED, with a reason.

3. Abstention: Two Layers

- **Layer 1, retrieval score:** if the best cosine score is below a threshold, the question is refused and the LLM is never called.
- **Layer 2, the LLM:** it returns `answerable: false` when the clauses do not contain the answer.
- An answer with no valid citation is also rejected.

4. Abstention Results

- 4 of 4 unanswerable questions (Q17 to Q20) were refused, all by the LLM.
- This includes near-miss questions. Q18 (defective product) was refused with a pointer to the Returns Policy in `MOP-3.5`.
- 1 of 16 answerable questions was refused wrongly: Q07, because `MOP-2.2` was missing from the top 3.

5. Faithfulness Results

- 15 answers were checked. Q07 and the 4 unanswerable questions have no answer to check.
- 14 are SUPPORTED, 1 is PARTIAL, none is UNSUPPORTED. All cited IDs are valid.
- **Q16 is PARTIAL.** The answer said the seller bears freight "only" for defects, misshipments or items not as described. Clause 8.3 lists when the seller pays, but never says the seller pays in no other case. The judge caught an over-claim.
- **Number check:** 1 of 15 flagged. In Q05 the value 2 is the model's own working (6 − 4 days), so it is not an error.

6. Abstention Threshold Study

- Off-topic questions (weather, cricket, cake) score 0.42 to 0.44, far below the lowest policy question (0.68).
- A threshold of about 0.55 would block off-topic questions without touching any of the 20 test questions.
- The current threshold of 0.35 is below all three off-topic scores, so it would block none of them.

CODE LINK: https://github.com/sandipanpaul21/olist-marketplace-intelligence/blob/main/notebooks/RAG/rag_05_faithfulness_abstention.ipynb

Policy RAG — Step 5: Faithfulness and Abstention (Part 2)

7. Final Numbers

- Retrieval (hybrid, top 3): all@3 0.94, all@5 1.00, any@1 0.81, MRR 0.91.
- Generation: 15 of 16 answerable questions answered, 4 of 4 unanswerable refused.
- Faithfulness: 14 of 15 SUPPORTED.
- Cited at least one correct clause: 16 of 16. This is a loose check, because it counts the refused Q07 and accepts one correct clause.

8. Limitations and Next Steps

- The judge and the answering model are the same model, which can be lenient. A different judge would be a stronger check.
- The off-topic test has only 3 questions.
- Next: set the threshold to about 0.55, re-run generation with dense + header and the top 5 clauses, and check that Q07 is fixed and the 4 unanswerable questions are still refused.

Background Concepts

1. Faithfulness vs Correctness: Faithful means the answer is supported by the clauses it was given. Correct means it matches the policy and the expected answer. An answer can be faithful and still wrong if the wrong clauses were retrieved.

2. LLM-as-Judge: A model grades another model's output against a rubric. It is cheap and scales, but it can be lenient toward its own answers and toward long answers. Use a different model as judge and spot-check some verdicts by hand.

3. Abstention (Selective Answering): The system declines to answer when the evidence is not enough. There are two kinds of error: a wrong refusal (an answerable question refused, 1 of 16 here) and a wrong answer (an unanswerable question answered, 0 of 4 here). Moving a threshold trades one for the other.

4. Choosing a Threshold: Pick it from data, in the gap between the groups it must separate, and report the size of that gap. A threshold tuned on the same questions it is tested on can look better than it is.

Notes: **Threshold:** The off-topic output shows that `MIN_DENSE_SCORE = 0.35` does nothing, even for off-topic questions. To change it, set `MIN_DENSE_SCORE = 0.55` in `src/policy_rag.py`. All 20 test questions score 0.684 or higher, so their results do not change.

CODE LINK: https://github.com/sandipanpaul21/olist-marketplace-intelligence/blob/main/notebooks/RAG/rag_05_faithfulness_abstention.ipynb